

Cypress Interview Preparation Handbook (2026 Edition)

Volume 2 - Component Testing, CI/CD Incidents, Performance Tuning, Live Coding, QA Architect Scenarios, Broken-Code Debugging

Prepared for Srikanth, SDET and Associate Architect QE

September 2026

Contents

How to Use This Volume	3
SECTION 11 — Component Testing (Real Scenarios)	4
Q1. A shared <code>FilterPanel</code> component works fine in isolation under Component Testing, but breaks only when rendered inside the real dashboard in E2E. How do you diagnose the gap between the two test types?	4
Q2. Your team wants to migrate 150 E2E tests that are really just testing isolated component behavior (form validation, button states) into Component Tests to cut CI time. How do you decide which ones qualify, and execute the migration safely?	5
Q3. How do you test a component that makes its own internal API call (not passed data via props) inside Component Testing, without hitting a real backend?	6
SECTION 12 — CI/CD (Real Pipeline Incidents)	6
Q4. Your Cypress Cloud parallel run reports “passed” overall, but you later discover one CI machine silently crashed mid-run and its specs never actually executed. How does this happen, and how do you guard against it?	6
Q5. Your Cypress suite passes consistently on Chrome in CI, but the team just added Firefox to the CI matrix and now 15 previously-stable tests fail only on Firefox. Triage this.	7
SECTION 13 — Performance & Stability (Real Tuning Decisions)	8
Q6. Your Cypress suite’s memory usage climbs across a long spec file with 60 tests, eventually crashing the browser tab partway through, even though each individual test looks clean. Diagnose it.	8
Q7. A specific spec file takes 4x longer in CI than it did last month, with no obvious test additions. How do you find what changed?	9
SECTION 14 — Real Coding Interview Questions (Live Coding Style)	10
Q8. Build a custom command that logs in as one of three roles using <code>cy.session()</code> with proper validation, and use it in a <code>describe</code> block.	10
Q9. This test intermittently fails with a “ <code>cy.click()</code> failed because this element is being covered by another element” error. Find the bug and fix it.	11
Q10. Write a test verifying a patient-ID column sorts correctly (numeric-aware, not lexicographic) after a header click, without hardcoding expected order.	12
SECTION 15 — QA Architect Scenario Questions (Real-Time System Design)	12

Q11. It's release day and your Cypress suite is stuck — 3 tests appear to be hanging indefinitely, blocking the whole cypress run from finishing. Walk through your next 2 hours vs. next 2 weeks.	12
Q12. Your company's product has both a Cypress-tested internal admin tool and a separate, newer customer-facing app the team wants built with Playwright for cross-browser needs. How do you architect testing strategy and CI across both without duplicating effort or confusing the team?	13
SECTION 16 — Scenario-Based Debugging (Broken Code, Live)	14
Q13. This test passes locally but fails in CI. Find the bug.	14
Q14. This test is supposed to verify only active cohorts appear in a filtered list, but it passes even when the filter logic is broken. Find the bug in the test.	14
Q15. Two spec files both use a shared commands.js helper that tracks "the last created cohort ID" in a module-level variable. Running the full suite, one file's test occasionally uses the wrong cohort. Diagnose from the code.	15

How to Use This Volume

This is **Volume 2** - the practical, real-time-facing companion to Cypress Volume 1 (Architecture, Custom Commands, Auth, Network Intercept, Retry-ability, Parallelism, Config, API Testing, Cross-Origin, Debugging). Every question here is built around an actual production scenario, a real incident pattern, or working-but-broken code - matching the same format used across both Playwright volumes.

Frequency legend: ★★★★★ Very Frequently Asked · ★★★★ Frequently Asked · ★★★ Occasionally Asked · ★★ Rare but High-Value

SECTION 11 — Component Testing (Real Scenarios)

Q1. A shared FilterPanel component works fine in isolation under Component Testing, but breaks only when rendered inside the real dashboard in E2E. How do you diagnose the gap between the two test types?

Difficulty: Senior / Staff **Asked In:** Microsoft, Adobe, ServiceNow **Frequency:** ★★★★★

Why Interviewer Asks This This exposes whether a candidate understands Component Testing's actual boundary — it validates a component's own logic in isolation, not its integration with real siblings, context providers, or app-level state, and conflating the two leads to false confidence.

Best Interview Answer Component Testing mounts FilterPanel with whatever props/context I choose to provide — if the real app wraps it in a Redux/Context provider supplying live data, and my component test mounted it with a simplified mock provider, any bug caused by the *real* provider's actual shape (a field the mock never included) will only surface in E2E. I diagnose by comparing exactly what context/props the component test mocked versus what the real app tree actually provides at that point, rather than assuming the component itself regressed — the bug is almost always in the fidelity of the test's mock, not the component's logic.

Real Project Example A FilterPanel component test mocked its CohortContext with a static { cohorts: [] } shape; the real app's context occasionally provided cohorts: null during a loading state — the component test never exercised that path, and only the full E2E flow (which hits the real loading sequence) caught the resulting crash.

Code Example

```
// Component test – mock context must match real shape's edge cases, not just the happy path
import { mount } from 'cypress/react';
import { CohortContext } from '../../src/context/CohortContext';
import { FilterPanel } from '../../src/components/FilterPanel';

it('handles null cohorts during loading without crashing', () => {
  mount(
    <CohortContext.Provider value={{ cohorts: null, loading: true }}>
      <FilterPanel />
    </CohortContext.Provider>
  );
  cy.get('[data-testid="filter-panel-skeleton"]').should('be.visible');
});
```

Common Mistakes Treating a passing Component Testing suite as proof the component is bug-free in all real contexts — it only proves correctness against the specific mocks you fed it.

Follow-Up Questions 1. How would you systematically audit component test mocks against real context shapes as the app evolves? 2. Where's the line between “this needs a component test” and “this needs an E2E test”?

QA Architect Tip Frame Component Testing and E2E as testing different risk categories entirely — unit-level correctness vs. integration correctness — and a passing suite in one says nothing about the other; this framing is the actual signal interviewers want.

Q2. Your team wants to migrate 150 E2E tests that are really just testing isolated component behavior (form validation, button states) into Component Tests to cut CI time. How do you decide which ones qualify, and execute the migration safely?

Difficulty: Staff / QA Architect **Asked In:** Walmart Global Tech, EPAM, Salesforce **Frequency:** ★★★

Best Interview Answer I audit each candidate test for whether its assertions depend on anything outside the component itself — real routing, real backend data, cross-component interaction. If a test only checks “does this form show a validation error when the email field is invalid,” that’s a pure component concern and migrates cleanly; if it checks “does submitting this form correctly navigate to and populate the next page,” that’s genuinely an integration/E2E concern and shouldn’t be forced into Component Testing. I migrate incrementally, keeping the old E2E test in place until the new component test is proven equivalent, then removing the E2E version — never deleting first and writing the replacement after.

Real Project Example Of 150 candidate tests, roughly 90 were genuinely pure-component form/UI-state checks that moved cleanly to Component Testing, cutting their collective CI contribution from several minutes of full-page loads down to seconds of isolated mounts; the remaining 60 tested real cross-page flows and correctly stayed as E2E.

Code Example

```
// Before – full E2E just to check client-side validation
it('shows error for invalid email', () => {
  cy.visit('/dashboards/cohort/new'); // full app load for a component-local concern
  cy.get('[data-testid="email-input"]').type('not-an-email');
  cy.get('[data-testid="submit-btn"]').click();
  cy.get('[data-testid="email-error"]').should('be.visible');
});

// After – component test, same assertion, no app load
import { mount } from 'cypress/react';
import { CohortForm } from '../src/components/CohortForm';

it('shows error for invalid email', () => {
  mount(<CohortForm onSubmit={cy.stub()} />);
  cy.get('[data-testid="email-input"]').type('not-an-email');
  cy.get('[data-testid="submit-btn"]').click();
  cy.get('[data-testid="email-error"]').should('be.visible');
});
```

Common Mistakes Migrating a test that actually depends on real routing or backend integration into Component Testing “because it looked simple” — losing real coverage while believing you’ve preserved it.

Follow-Up Questions 1. How would you catch a case where a migrated component test’s mock silently drifts from the real prop shape over time? 2. How do you communicate to the team which category a new test should default to, going forward?

QA Architect Tip State the audit question explicitly: “does this assertion depend on anything the component itself doesn’t own?” — a clean litmus test interviewers want to hear articulated, not just a vague sense of “this one felt simple enough.”

Q3. How do you test a component that makes its own internal API call (not passed data via props) inside Component Testing, without hitting a real backend?

Difficulty: Senior **Asked In:** ServiceNow, Optum, Meesho **Frequency:** ★★★★★

Best Interview Answer Component Testing still runs in a real browser, so `cy.intercept()` works exactly the same way it does in E2E — I intercept the component's internal fetch call before mounting, letting the component's real internal logic run against a controlled, stubbed response, rather than trying to mock the fetch function itself at the JavaScript module level.

Code Example

```
import { mount } from 'cypress/react';
import { CohortSummaryWidget } from '../../src/components/CohortSummaryWidget';

it('renders member count from its own internal fetch', () => {
  cy.intercept('GET', '/api/cohorts/123/summary', {
    body: { memberCount: 250 },
  }).as('getSummary');

  mount(<CohortSummaryWidget cohortId="123" />);

  cy.wait('@getSummary');
  cy.get('[data-testid="member-count"]').should('have.text', '250 members');
});
```

Common Mistakes Reaching for a JS-level mocking library (like `sinon` stubbing `fetch` globally) when `cy.intercept()` already handles this at the network layer, consistently with how E2E tests do it — using two different mocking mechanisms across component vs. E2E tests adds unnecessary cognitive overhead for the team.

Follow-Up Questions 1. How would you test the component's loading and error states for that same internal fetch? 2. What's the trade-off of a component owning its own data-fetching vs. receiving data purely via props, from a testability standpoint?

QA Architect Tip Point out explicitly that components owning their own fetching are inherently harder to test in isolation than pure “dumb” components receiving props — a good architect sometimes recommends a refactor toward prop-driven components specifically to improve testability, not just writes around the existing design.

SECTION 12 — CI/CD (Real Pipeline Incidents)

Q4. Your Cypress Cloud parallel run reports “passed” overall, but you later discover one CI machine silently crashed mid-run and its specs never actually executed. How does this happen, and how do you guard against it?

Difficulty: Staff / QA Architect **Asked In:** Walmart Global Tech, JP Morgan, ServiceNow **Frequency:** ★★★★★

Why Interviewer Asks This This is a real, high-stakes failure mode — a release shipping on a false-green signal — and tests whether the candidate has actually operated Cypress Cloud in production, not just read about `--parallel`.

Best Interview Answer If a machine crashes (OOM, network drop, runner eviction) before it reports any results back to Cypress Cloud, and the CI orchestration layer (GitHub Actions, Jenkins) doesn't treat that job's failure as a pipeline failure — for instance, if the job step itself is marked `continue-on-error` or the crash happens in a way that doesn't propagate a non-zero exit code to the orchestrator — the overall pipeline can report success while a whole slice of specs never ran. I guard against this two ways: first, the CI job itself must fail loudly (non-zero exit, no swallowed errors) if the Cypress process dies unexpectedly; second, I cross-check Cypress Cloud's dashboard for the *expected total spec count* against what actually ran for that build ID, ideally as an automated post-run check, not a manual glance.

Real Project Example A CI runner got evicted mid-run by the cloud provider's spot-instance reclaiming policy; the orchestrating pipeline's overall status showed green because the *other 7* machines all passed, and nobody had a check comparing "specs expected" vs. "specs actually recorded" — a scripted post-run assertion comparing `cypress-cloud` API's reported spec count against the known total spec count in the repo would have caught this immediately.

Code Example

```
# Post-run guard: compare expected spec count against what Cypress Cloud actually recorded
EXPECTED=$(find cypress/e2e -name "*.cy.*" | wc -l)
ACTUAL=$(curl -s -H "Authorization: Bearer $CYPRESS_CLOUD_TOKEN" \
  "https://cloud.cypress.io/api/runs/${CI_BUILD_ID}" | jq '.specs | length')

if [ "$ACTUAL" -lt "$EXPECTED" ]; then
  echo "MISMATCH: expected $EXPECTED specs, only $ACTUAL recorded – failing pipeline"
  exit 1
fi
```

Common Mistakes Trusting the CI orchestrator's green checkmark alone without ever validating that it actually reflects the intended total scope of the run — a distributed run silently losing a slice of coverage is a real, recurring incident category at scale.

Follow-Up Questions 1. How would you set an alert for this proactively rather than discovering it after the fact? 2. What's the difference in blast radius between a single spec file failing vs. an entire machine's specs never running at all?

QA Architect Tip This is the Cypress-specific version of "don't trust a green build without verifying it actually covers what you think it covers" — the same discipline applies to any distributed test execution system, Cypress Cloud or otherwise.

Q5. Your Cypress suite passes consistently on Chrome in CI, but the team just added Firefox to the CI matrix and now 15 previously-stable tests fail only on Firefox. Triage this.

Difficulty: Senior / Staff **Asked In:** Microsoft, Adobe, Flipkart **Frequency:** ★★★★★

Best Interview Answer Cross-browser failures concentrated in a specific new browser almost always point to either genuine rendering/timing differences (Firefox's event loop and paint timing differ from Chromium's) or Cypress-Firefox-specific quirks (historically, some Cypress features and plugins have had partial Firefox support). I triage by grouping the 15 failures by symptom rather than fixing them one-by-one blindly — if most share a pattern (e.g., all involve a specific animation or a specific `cy.intercept()` usage), that points to one systemic cause rather than 15 unrelated bugs, and I check Cypress's own browser-compatibility documentation and changelog for known Firefox-specific caveats on that feature before assuming it's an app bug.

Real Project Example 12 of the 15 Firefox-only failures shared a common root cause: a CSS transition duration Firefox rendered slightly differently, causing Cypress's stability-adjacent checks to catch the button mid-animation more often than in Chromium — adding an explicit `.should('not.have.class', 'transitioning')` before interacting fixed the whole cluster at once, rather than being 12 separate fixes.

Code Example

```
// Cross-browser-safe stability check, not relying on Chromium-specific timing tolerance
cy.get('[data-testid="save-btn"]')
  .should('not.have.class', 'transitioning')
  .should('be.visible')
  .click();
```

Common Mistakes Fixing each of the 15 failing tests independently with test-specific waits/retries instead of first grouping by symptom to find the one or two systemic causes likely responsible for most of them.

Follow-Up Questions 1. How would you decide whether a Firefox-specific failure is a real app bug worth fixing vs. a test-authoring gap? 2. What's your policy on running full regression against every configured browser on every PR vs. a subset?

QA Architect Tip Grouping failures by symptom before touching any code, rather than fixing tests one at a time, is exactly the structured triage instinct Staff interviewers are scoring for — say it out loud as your first step.

SECTION 13 — Performance & Stability (Real Tuning Decisions)

Q6. Your Cypress suite's memory usage climbs across a long spec file with 60 tests, eventually crashing the browser tab partway through, even though each individual test looks clean. Diagnose it.

Difficulty: Senior / Staff **Asked In:** Microsoft, Walmart Global Tech, Optum **Frequency:** ★★★★★

Best Interview Answer Since Cypress runs all tests in a spec file inside the same browser tab/process (unlike Playwright's fresh context per test), anything that accumulates across tests without being cleaned up — event listeners registered via `cy.intercept()` that aren't scoped per-test, DOM nodes from a component that doesn't unmount cleanly between mounts in Component Testing, or a growing in-memory cache the app itself builds up — has nowhere to go until the whole spec file finishes and the tab reloads for the next file. I isolate by bisecting the spec file (running the first 30 tests, then the last 30, checking where memory growth actually concentrates) rather than assuming it's evenly distributed across all 60.

Real Project Example A spec file mounting a heavy visualization component 60 times in Component Testing leaked chart library instances that weren't being destroyed between mounts — the visualization library's own cleanup method needed to be called explicitly in an `afterEach`, since Cypress's `testIsolation` resets storage/cookies but doesn't know to call a third-party library's specific teardown API.

Code Example

```
afterEach(() => {
  cy.window().then((win) => {
```

```
// Explicit cleanup for a library Cypress's isolation can't know about
if (win.chartInstance) {
  win.chartInstance.destroy();
}
});
});
```

Common Mistakes Assuming `testIsolation: true` handles all cleanup — it resets cookies/localStorage/page navigation, not arbitrary third-party library instances or event listeners the app itself created.

Follow-Up Questions 1. How would you profile exactly which objects are accumulating using Chrome DevTools memory snapshots across a repro run? 2. Would splitting one 60-test spec file into several smaller files help here, and why?

QA Architect Tip Splitting a long spec file into several smaller ones is often a legitimate practical mitigation even before finding the root leak — each spec file gets a fresh tab/process in Cypress, effectively resetting accumulated memory — mention this as a pragmatic stopgap alongside the real fix.

Q7. A specific spec file takes 4x longer in CI than it did last month, with no obvious test additions. How do you find what changed?

Difficulty: Senior **Asked In:** ServiceNow, Salesforce, Razorpay **Frequency:** ★★★

Best Interview Answer I don't guess — I pull Cypress Cloud's (or the CI log's) historical timing data for that specific spec file over the past month and look for a step-change point rather than gradual drift, which usually correlates cleanly with a specific merged PR. Common real causes: a new `cy.intercept()` added without a `delay` override now waiting on the full real network round-trip where it used to be instant; a shared `beforeEach` in that file starting to do more setup than before; or an app-side regression (a genuinely slower page load) that the test is faithfully reporting, not a test-authoring problem at all.

Real Project Example A spec file's 4x slowdown traced to a `beforeEach` that started calling a newly-added `cy.session()`-wrapped login, but the `validate` callback itself was hitting a slow, unrelated real endpoint on every single test instead of a cheap health-check — narrowing the `validate` check to the lightest possible endpoint cut the file's runtime back to baseline.

Code Example

```
# Pull historical spec-level timing from Cypress Cloud API to find the exact regression point
curl -s -H "Authorization: Bearer $CYPRESS_CLOUD_TOKEN" \
  "https://cloud.cypress.io/api/projects/$PROJECT_ID/runs?spec=cohort-explorer.cy.ts" \
  | jq '.runs[] | {createdAt, duration}'
```

Common Mistakes Assuming the slowdown must be in the test code itself and never checking whether the *application* actually got slower — a regression test suite reporting a real app performance regression is doing its job correctly, not misbehaving.

Follow-Up Questions 1. How would you distinguish “the app got slower” from “the test got slower” definitively? 2. What ongoing monitoring would catch this kind of gradual or step-change regression automatically, rather than someone noticing manually a month later?

QA Architect Tip Say explicitly that historical, spec-level timing data (not just “the whole suite feels slower”) is what actually pinpoints the regression — most teams don't retain or check this data until they're forced to during an incident.

SECTION 14 — Real Coding Interview Questions (Live Coding Style)

Q8. Build a custom command that logs in as one of three roles using `cy.session()` with proper validation, and use it in a describe block.

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, JP Morgan **Frequency:** ★★★★★

Why Interviewer Asks This The Cypress equivalent of the Playwright role-fixture exercise — combines `cy.session()`, custom commands, and TypeScript typing in one gradeable exercise.

Best Interview Answer (spoken while coding) “I’ll key the session cache by role name, do the actual login via `cy.request()` so it’s fast and independent of the UI, and add a `validate` callback so a stale cached session gets transparently refreshed rather than silently reused.”

Code Example

```
// cypress/support/commands/auth.ts
type Role = 'admin' | 'analyst' | 'auditor';

Cypress.Commands.add('loginAs', (role: Role) => {
  cy.session(role, () => {
    cy.request('POST', '/api/auth/login', {
      username: Cypress.env(`${role.toUpperCase()}_USER`),
      password: Cypress.env(`${role.toUpperCase()}_PASS`),
    }).then((response) => {
      window.localStorage.setItem('authToken', response.body.token);
    });
  }, {
    validate: () => {
      cy.request({ url: '/api/session/check', failOnStatusCode: false })
        .its('status').should('eq', 200);
    },
  });
});
```

```
// cypress/support/index.d.ts
declare namespace Cypress {
  interface Chainable {
    loginAs(role: 'admin' | 'analyst' | 'auditor'): Chainable<void>;
  }
}
```

```
// cypress/e2e/admin/user-management.cy.ts
describe('admin-only actions', () => {
  beforeEach(() => {
    cy.loginAs('admin');
    cy.visit('/admin/users');
  });

  it('admin can deactivate a user', () => {
    cy.get('[data-testid="user-row-jane"]').find('[data-testid="deactivate-btn"]').click()
    cy.get('[data-testid="toast-success"]').should('have.text', 'User deactivated');
  });
});
```

```
});
});
```

Common Mistakes Forgetting the `.d.ts` augmentation (TypeScript errors on every call site) or omitting `validate` (stale sessions silently reused after token expiry).

Follow-Up Questions 1. How would you extend this to support a tenant dimension alongside role, the way the Playwright handbook covers multi-tenant auth? 2. What happens if two tests in the same file call `cy.loginAs()` with different roles — does session caching handle that correctly?

QA Architect Tip Narrate “cache key = role, validated on reuse, real login only once per role per run” while coding — that’s the exact phrase interviewers are listening for.

Q9. This test intermittently fails with a “cy.click() failed because this element is being covered by another element” error. Find the bug and fix it.

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, Swiggy **Frequency:** ★★★★★

Given (broken) code:

```
it('applies filter and shows updated results', () => {
  cy.visit('/dashboards/cohort-explorer');
  cy.get('[data-testid="age-range-input"]').type('40-60');
  cy.get('[data-testid="apply-filters-btn"]').click();
  cy.get('[data-testid="cohort-count"]').should('contain.text', 'members');
});
```

What’s wrong / why intermittent The app shows a loading overlay covering the filter panel briefly after typing (e.g., a debounced live-preview fetch), and depending on timing, the `.click()` sometimes lands while that overlay is still on top of the button — Cypress correctly refuses to click something visually obscured rather than silently force-clicking through it. This is intermittent because the overlay’s timing depends on network/render speed, which varies between local and CI.

Fix

```
it('applies filter and shows updated results', () => {
  cy.visit('/dashboards/cohort-explorer');
  cy.get('[data-testid="age-range-input"]').type('40-60');
  cy.get('[data-testid="loading-overlay"]').should('not.exist'); // wait for overlay to clear
  cy.get('[data-testid="apply-filters-btn"]').click();
  cy.get('[data-testid="cohort-count"]').should('contain.text', 'members');
});
```

Common Mistakes Reaching for `.click({ force: true })` as the fix — this bypasses the actionability check entirely and can mask a real bug where the overlay never actually clears, silently clicking through it instead of surfacing the real issue.

Follow-Up Questions 1. Why is `{ force: true }` almost always the wrong fix here, specifically? 2. How would you find the exact element covering the button using Cypress’s own error output or DevTools?

QA Architect Tip Say explicitly that Cypress’s refusal to click an obscured element is a *feature*, not a bug getting in your way — the fix should address why it’s obscured, never suppress the

check.

Q10. Write a test verifying a patient-ID column sorts correctly (numeric-aware, not lexicographic) after a header click, without hardcoding expected order.

Difficulty: Senior **Asked In:** ServiceNow, Optum, Meesho **Frequency:** ★★★★★

Best Interview Answer Same trap and same fix as the equivalent Playwright question — compute the expected order programmatically from whatever data is actually rendered, using the app’s intended sort semantic, rather than asserting a hardcoded literal sequence tied to today’s seed data.

Code Example

```
it('patient ID column sorts numerically, not lexicographically', () => {
  cy.visit('/dashboards/patients');
  cy.get('[data-testid="patient-id-header"]').click();

  cy.get('[data-testid="patient-id-cell"]').then(($cells) => {
    const ids = [...$cells].map((el) => el.textContent.trim());
    const extractNumber = (id) => parseInt(id.replace('P-', ''), 10);
    const expected = [...ids].sort((a, b) => extractNumber(a) - extractNumber(b));
    expect(ids).to.deep.equal(expected);
  });
});
```

Common Mistakes Hardcoding ['P-9', 'P-23', 'P-102'] tied to today’s specific seeded data — breaks the moment test data changes, even if the sort logic is still correct.

Follow-Up Questions 1. How would you test the descending sort on a second click? 2. Why is this an explicit assertion (expect) here rather than an implicit .should() — is that the right choice?

QA Architect Tip Note that this is a case where an explicit assertion inside .then() is actually correct — you need the full array captured before computing and comparing, and retry-ability doesn’t meaningfully apply once you’re doing a one-time computed comparison; know when the “prefer .should()” rule doesn’t apply.

SECTION 15 — QA Architect Scenario Questions (Real-Time System Design)

Q11. It’s release day and your Cypress suite is stuck — 3 tests appear to be hanging indefinitely, blocking the whole cypress run from finishing. Walk through your next 2 hours vs. next 2 weeks.

Difficulty: Staff / QA Architect **Asked In:** Walmart Global Tech, JP Morgan, ServiceNow **Frequency:** ★★★★★

Best Interview Answer Next 2 hours: since Cypress runs specs sequentially in one process by default, a hanging test blocks everything after it in that spec file and potentially the whole run depending on CI timeout configuration. I’d check CI logs for exactly which command it’s

stuck on (Cypress logs each command as it runs), kill the job, temporarily skip or quarantine just the offending test(s) with `it.skip()`, and re-run to get a trustworthy signal on everything else — then make a risk-based release call with the release owner on the quarantined area specifically, not unilaterally. Next 2 weeks: root-cause why those tests hang — usually a `cy.wait('@alias')` on an intercept alias that never fires because the request pattern doesn't match, or a retry-ability loop waiting on an assertion that can genuinely never become true — and add a global safety net (`defaultCommandTimeout` tuned sensibly, plus CI-level job timeouts) so a single hanging test can never again block an entire release pipeline indefinitely.

Real Project Example Three tests hung because a recent app change altered an API endpoint's URL pattern slightly, and `cy.wait('@getCohorts')` was waiting on an intercept alias that could now never match — quarantining those three, shipping on the rest, then fixing the intercept pattern and un-quarantining them the same day was the actual real-world resolution.

Common Mistakes Letting the entire CI job time out and fail with no actionable signal, instead of proactively identifying and isolating the specific hanging tests first to salvage a trustworthy signal on everything else.

Follow-Up Questions 1. How would you configure CI-level timeouts specifically to prevent one hanging test from consuming the whole job's time budget? 2. How do you decide what counts as “safe to ship” when a nontrivial area is quarantined under deadline pressure?

QA Architect Tip Same “stop the bleeding, then fix the disease” framing as the Playwright handbook's equivalent question — Staff interviewers want to see that operational instinct regardless of which tool triggered the incident.

Q12. Your company's product has both a Cypress-tested internal admin tool and a separate, newer customer-facing app the team wants built with Playwright for cross-browser needs. How do you architect testing strategy and CI across both without duplicating effort or confusing the team?

Difficulty: Staff / QA Architect **Asked In:** EPAM, Deloitte Digital, Adobe **Frequency:** ★★★★★

Best Interview Answer I don't force one tool onto both applications just for consistency — each keeps the tool that fits its actual requirements (Cypress for the Chromium-only internal admin tool where its lighter setup and existing suite have real value; Playwright for the customer-facing app needing Safari/WebKit and multi-tab support). What *does* need unifying: shared conventions across both — the same tagging scheme for smoke/regression, the same reporting destination so leadership sees one dashboard rather than two disconnected ones, and shared documentation on which tool owns which repo so new engineers aren't confused about where to add a test. I explicitly avoid a “rewrite everything in one tool” project unless there's a concrete technical driver, since that's often solving a consistency preference at real engineering cost.

Real Project Example Two suites — 400 Cypress tests for an internal admin tool, 200 Playwright tests for the external product — reported into the same internal test-health dashboard via a small script normalizing both tools' JSON output into one shared schema, giving leadership one unified pass-rate view without merging the actual test codebases.

Common Mistakes Treating tool consistency as inherently more valuable than fit-for-purpose tool choice — forcing a rewrite of a stable, working Cypress suite into Playwright purely for consistency, at real cost, with no corresponding capability gap being solved.

Follow-Up Questions 1. How would you onboard a new engineer who needs to contribute to

both suites? 2. What would change your recommendation toward unifying onto one tool after all?

QA Architect Tip Explicitly reject “one tool for everything” as a default — the mature architect answer optimizes per-application fit and unifies only the parts that benefit from unification (reporting, conventions), not the tooling itself.

SECTION 16 — Scenario-Based Debugging (Broken Code, Live)

Q13. This test passes locally but fails in CI. Find the bug.

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, ServiceNow **Frequency:** ★★★★★

Given (broken) code:

```
it('exports report and shows success toast', () => {
  cy.visit('/dashboards/cohort/123');
  cy.get('[data-testid="export-btn"]').click();
  cy.wait(2000);
  cy.get('[data-testid="export-success-toast"]').should('be.visible');
});
```

What's wrong A fixed 2-second `cy.wait(ms)` assumes the export always completes within that window — under CI resource contention or real backend load, it can take longer, and the assertion runs before the toast ever appears. This is the exact same actionability-vs-business-readiness gap covered throughout the Playwright handbooks, expressed in Cypress.

Fix

```
it('exports report and shows success toast', () => {
  cy.visit('/dashboards/cohort/123');
  cy.intercept('POST', '/api/cohorts/123/export').as('exportRequest');
  cy.get('[data-testid="export-btn"]').click();
  cy.wait('@exportRequest'); // wait on the actual network event, not a guessed duration
  cy.get('[data-testid="export-success-toast"]', { timeout: 10000 }).should('be.visible');
});
```

Follow-Up Questions 1. Why is `cy.wait('@alias')` fundamentally different from `cy.wait(ms)` in terms of what it guarantees? 2. What if the toast briefly appears and disappears — would this fix reliably catch it?

QA Architect Tip Say plainly: “if I see a bare `cy.wait(1000)` in a PR review where an intercept alias or a `.should()` would work instead, that’s a request-changes” — matches the Playwright handbook’s identical stance on `waitForTimeout`.”

Q14. This test is supposed to verify only active cohorts appear in a filtered list, but it passes even when the filter logic is broken. Find the bug in the test.

Difficulty: Senior **Asked In:** EPAM, Optum, Deloitte Digital **Frequency:** ★★★

Given (broken) code:

```
it('only active cohorts are shown', () => {
  cy.createTestCohort({ name: 'Active One', status: 'active' });
  cy.createTestCohort({ name: 'Archived One', status: 'archived' });

  cy.visit('/dashboards/cohorts?filter=active');
  cy.get('[data-testid="cohort-row"]').should('have.length', 1);
});
```

What's wrong Asserting an absolute count in a shared staging environment is fragile — other tests' leftover or concurrently-created cohorts can make the count wrong for reasons unrelated to the filter logic under test, and worse, a coincidental match can make a genuinely broken filter appear to pass.

Fix

```
it('only active cohorts are shown, archived ones are excluded', () => {
  const suffix = Date.now();
  cy.createTestCohort({ name: `Active-${suffix}`, status: 'active' }).then((activeId) => {
    cy.createTestCohort({ name: `Archived-${suffix}`, status: 'archived' }).then((archivedId) => {
      cy.visit('/dashboards/cohorts?filter=active');
      cy.get(`[data-testid="cohort-row-${activeId}"]`).should('exist');
      cy.get(`[data-testid="cohort-row-${archivedId}"]`).should('not.exist');

      cy.deleteTestCohort(activeId);
      cy.deleteTestCohort(archivedId);
    });
  });
});
```

Follow-Up Questions 1. Why is asserting presence/absence of specific known entities more robust than an absolute count here? 2. How would this need to change for a genuine pagination-count test?

QA Architect Tip Same principle as the Playwright handbooks: test *behavior against known, owned data*, never a snapshot of shared environment state.

Q15. Two spec files both use a shared `commands.js` helper that tracks “the last created cohort ID” in a module-level variable. Running the full suite, one file’s test occasionally uses the wrong cohort. Diagnose from the code.

Difficulty: Senior / Staff **Asked In:** Walmart Global Tech, JP Morgan, PhonePe **Frequency:** ★★★

Given (broken) code:

```
// cypress/support/commands/cohorts.js
let lastCreatedCohortId;

Cypress.Commands.add('createTestCohort', (params) => {
  return cy.request('POST', '/api/cohorts', params).then((response) => {
    lastCreatedCohortId = response.body.id;
    return response.body.id;
  });
});
```

```
});  
});
```

```
Cypress.Commands.add('getLastCohortId', () => lastCreatedCohortId);
```

What's wrong `lastCreatedCohortId` is module-level state shared across every spec file in the same Cypress run — if two spec files both create cohorts and one file's test calls `cy.getLastCohortId()` expecting *its own* most recent creation, but another file's test created a cohort afterward in the shared execution order, the wrong ID is returned. This is the same class of bug as the Playwright handbook's module-level shared-state example — Cypress isn't immune to it just because it lacks Playwright's parallel-worker model.

Fix

```
// Don't rely on shared module state at all – pass the ID explicitly through the test  
Cypress.Commands.add('createTestCohort', (params) => {  
  return cy.request('POST', '/api/cohorts', params).its('body.id');  
});
```

```
// spec file – capture and use the ID directly, no shared module state involved  
it('shows correct data for a specific cohort', () => {  
  cy.createTestCohort({ name: 'test-cohort' }).then((cohortId) => {  
    cy.visit(`/dashboards/cohort/${cohortId}`);  
    // ... assertions using cohortId directly, not a shared "last created" variable  
  });  
});
```

Follow-Up Questions 1. Why does removing the shared variable entirely, rather than resetting it in a hook, fully eliminate the bug class? 2. How would you audit an existing `commands.js` file for other instances of this same pattern?

QA Architect Tip The stronger answer removes the “convenience” helper (`getLastCohortId`) entirely rather than trying to reset it correctly between tests — the safest fix to shared mutable state is usually not having it at all, not managing it more carefully.
